Módulo 2: Gestión de Proyectos (Estilo Kanban)

Estructura de Tableros y Columnas:

- **Objetivo:** CRUD completo con rutas anidadas para reflejar jerarquía: `/api/boards/:boardId/columns/:columnId/tickets`.
- **Buenas Prácticas:** Diseño RESTful estricto, verificación de existencia del recurso padre antes de operar sobre el hijo, y borrado en cascada (eliminar un tablero elimina sus tickets).
- **No es obligatorio** implementar autenticación en esta etapa.

Objetivo: Desarrollar la API RESTful que dará soporte a la estructura principal de la aplicación. Deberán implementar un CRUD completo utilizando rutas anidadas que reflejen la relación Padre-Hijo-Nieto entre Tableros, Columnas y Tickets.



1. Especificación de Endpoints (Contrato de la API)

El sistema debe exponer exactamente la siguiente estructura de rutas. No se permite el uso de rutas planas (ej. `/api/tickets`) para la creación, ya que todo ticket debe nacer estrictamente dentro del contexto de una columna y un tablero.

Método	Endpoint	Acción	Código de Éxito
POST	/api/boards	Crea un nuevo tablero.	201 Created
GET	/api/boards/:boardId	Obtiene un tablero con sus columnas pobladas.	200 OK
POST	/api/boards/:boardId/columns	Agrega una columna a un tablero específico.	201 Created
DELETE	/api/boards/:boardId/columns/:columnId	Elimina una columna.	204 No Content
POST	/api/boards/:boardId/columns/:columnId /tickets	Crea un ticket dentro de una columna específica.	201 Created
PATCH	/api/boards/:boardId/columns/:columnId /tickets/:ticketId	Mueve un ticket o actualiza su contenido.	200 OK

2. Requerimientos Técnicos y Reglas de Negocio

Para aprobar el ejercicio, el código debe implementar las siguientes validaciones y buenas prácticas en el backend:

- **Verificación estricta de existencia (Parent Check):** Antes de crear una Columna, el controlador debe consultar a la base de datos si el `boardId` existe. Si no existe, debe abortar la operación inmediatamente y retornar un `404 Not Found` (no un `500` ni un error genérico de Mongoose). La misma regla aplica al crear un Ticket respecto a su `columnId`.
- **Aislamiento de Rutas (Middleware):** Si un cliente intenta acceder a `/api/boards/123/columns/456/tickets`, el sistema debe validar que la columna `456` efectivamente pertenece al tablero `123`. Si la columna existe pero pertenece a otro tablero, la petición debe ser rechazada con un `400 Bad Request` o `404 Not Found`.
- **Respuestas HTTP Semánticas:**
    - `400 Bad Request` si el payload falla la validación (ej. falta el título del ticket).

- `404 Not Found` si el ID proporcionado tiene formato de `ObjectId` válido pero no existe en la base de datos.
- `400 Bad Request` (o error de casteo capturado) si el ID proporcionado no tiene el formato de 24 caracteres hexadecimales de MongoDB.

3. Entregables Esperados

1. **Repositorio del proyecto:** público, con README.md completo, conventional commits y archivos de ejemplo (.env.example)
2. **Modelos:** Archivos separados para `Board.js`, `Column.js` y `Ticket.js` con sus respectivos esquemas y referencias (`ref`).
3. **Controladores y Rutas:** Módulos separados respetando el principio de responsabilidad única.
4. **Pruebas en tiempo real o vídeo** con registro de las operaciones utilizando Postman, ThunderClient u otra herramienta similar
5. Se valorará el despliegue del backend.

Desarrollo Basado en Especificaciones (SDD)

Referencia: https://agents.md/

1. Centralización de Reglas en `AGENTS.md`

En lugar de repetir las reglas arquitectónicas en cada prompt, el contexto global debe residir en un archivo de configuración del proyecto (ej. `AGENTS.md` o `.cursorrules`).

- **Definir el Stack Estricto:** Especificar las versiones y librerías (ej. Node.js 20, Mongoose 8, Express 4).
- **Fijar Patrones de Diseño:** Indicar explícitamente que se debe usar el patrón de Responsabilidad Única (separando Modelos, Controladores y Rutas) y no mezclar lógica de base de datos en los archivos de rutas.
- **Manejo de Errores Global:** Establecer el estándar de respuestas HTTP esperadas (ej. siempre retornar `{ error: "mensaje" }` en caso de fallos).

2. Aplicación de "Límites Negativos" (Negative Boundaries)

Los agentes de IA tienden a tomar atajos o asumir comportamientos por defecto si no se los restringe. Una especificación sólida debe incluir explícitamente lo que **no** se debe hacer.

- **Ejemplo para el Kanban:** *"NO utilices arrays simples para anidar tickets dentro de las columnas si superan los 100 ítems; utiliza referencias (ObjectIds) de Mongoose."*
- *"NO devuelvas un error 500 genérico si el `boardId` no existe; captura la validación y retorna un 404."*
- *"NO utilices `findByIdAndDelete` sin antes disparar los hooks de Mongoose para asegurar el borrado en cascada."*

3. Desarrollo Invertido: Pruebas y Contratos Primero

El agente debe conocer cómo será evaluado antes de escribir la primera línea de lógica de negocio.

- **Proveer la Tabla de Endpoints:** Entregar al agente la tabla exacta de rutas, métodos y códigos de estado (como la del ejercicio anterior) como la "fuente de la verdad".
- **Generar Colecciones de Prueba Iniciales:** Pedirle al agente que primero genere el archivo `thunder-collection.json` basado en la especificación. Al tener las pruebas listas en ThunderClient, los estudiantes pueden iterar la generación del código hasta que todas las peticiones pasen a verde.

4. Partición Generativa (Evitar el "Hazlo Todo")

Pedirle a un agente que genere los tres modelos (Board, Column, Ticket) y todos sus controladores en un solo prompt suele generar código espagueti o pérdida de contexto. La especificación debe ejecutarse paso a paso:

1. **Fase 1 (Datos):** *"Basado en la especificación, genera solo los esquemas de Mongoose con sus referencias y el middleware `pre('deleteOne')` para el borrado en cascada."* -> **Revisar y validar.**
2. **Fase 2 (Validación):** *"Genera el middleware que verifique la existencia del recurso padre (Parent Check) para las rutas anidadas."* -> **Revisar y validar.**
3. **Fase 3 (Controladores):** *"Implementa el controlador para `POST /api/boards/:boardId/columns/:columnId/tickets` utilizando los modelos y middlewares previamente generados."*

5. Exigir Idempotencia y Manejo de Concurrencia

En el SDD, la resiliencia de la API debe estar especificada. Al pedirle al agente que construya los endpoints de actualización (como el `PATCH` para mover un ticket de columna), la instrucción debe exigir prácticas de control.

Idempotencia: en programación es la propiedad de una operación que se puede realizar varias veces y produce el mismo resultado que si se hiciera una sola vez, sin causar efectos secundarios adicionales

- **Prompt de Refuerzo:** *"Asegúrate de que el endpoint de actualización del ticket sea idempotente. Si el cliente envía la misma petición dos veces por un fallo de red, el estado final en la base de datos debe ser el mismo sin duplicar datos ni corromper el orden."*